

MATA54 - Estruturas de Dados e Algoritmos II

Arquivos Sequenciais

George Lima e Flávio Assis

IC - Instituto de Computação

Última atualização: 1 de outubro de 2024

Organização de Arquivos Sequenciais

Arquivo:

Sequência de **registros**

Registro:

Item de dado contendo informação de um elemento da aplicação

- ▶ Um registro contém um conjunto de **campos**: campo 1, campo 2, ..., campo m

Registros estão dispostos em uma sequência de endereços físicos. Acesso aos mesmos pode ser **sequencial** ou **direto**.

Endereços:	1	2	3	...	n
Registros:	r_1	r_2	r_3	...	r_n

Chave Primária e Secundária

Uma **chave primária** é um campo (ou conjunto de campos) cujo valor identifica um registro de forma única.

Qualquer combinação de campos que não identifique um registro de forma única é chamado de uma **chave secundária**

Exemplos de chave primária:

- ▶ CPF em registros sobre pessoa física
- ▶ suponha que cada departamento em uma banco de dados de uma empresa possua um identificador único e que cada funcionário de um departamento possua um identificador único **apenas em seu departamento**. O identificador do departamento juntamente com o identificador do funcionário é uma chave primária

Qual a importância da definição de chave primária e secundária?

A busca por uma chave primária resulta em no máximo um registro como resposta. A busca por chave secundária pode retornar potencialmente vários registros como resposta

Um arquivo pode ser organizado de forma a otimizar o tipo de resposta

Busca por Chave Primária

A **busca por chave primária** consiste em verificar se existe ou não registro no arquivo cujo valor de chave primária é igual a um determinado valor desejado k .

Se houver, a busca deve retornar uma indicação de qual é este registro.

Caso contrário, deve informar que tal registro não existe.

Quando não houver ambigüidade, usaremos apenas **chave** para denotar chave primária.

Notação: denotaremos por $r.k$ o valor da chave primária de um registro r qualquer; $A[i].k$ representará a chave do registro r_i , de índice i , pertencente ao arquivo A

Busca por Chave Primária: Busca Sequencial

Estratégia de busca

A partir do primeiro registro, compara-se registro após registro, na sequência do arquivo, buscando pela chave primária do registro que é igual ao valor desejado k .

Algorithm 1: Busca sequencial

entrada: Arquivo A com n registros; valor k a ser procurado

saida : Índice do registro no arquivo cuja chave é k (caso exista) ou -1 (caso contrário)

```
1  $i \leftarrow 1$ ;  
2 while  $(i \leq n) \wedge (A[i].k \neq k)$  do  
3    $i \leftarrow i + 1$ ;  
4 if  $i \leq n$  then return  $i$ ;  
5 return  $-1$ ;
```

Complexidade - busca sequencial

Algoritmo	Busca com Sucesso		Busca sem Sucesso
	Melhor Caso	Pior Caso	
Busca Sequencial (não ordenado)	$O(1)$	$O(n)$	$O(n)$

Complexidade - Busca Sequencial (caso médio)

- ▶ Como se calcula a média dos casos? Quantos casos existem?
- ▶ Qual o peso de cada caso?

Modelo probabilístico

- ▶ A probabilidade de se buscar um registro existente no arquivo A é p
- ▶ Todos os registros de A têm a mesma probabilidade de serem buscados

$$\forall i, j \in \{1, \dots, n\}, \forall k \in A (\Pr\{k = A[i].k\} = \Pr\{k = A[j].k\} = p/n)$$

- ▶ **De fato não se conhece p ou a probabilidade de um registro em A ser buscado**

O custo médio C_n para se buscar um registro qualquer num arquivo com n registros é a média para: buscar qualquer registro $A[i]$, que tem custo proporcional a i ; e buscar um registro inexistente, que tem custo proporcional a n

$$C_n = \sum_{i=1}^n \frac{p}{n} \times i + (1 - p) \times n = \frac{p(n+1)}{2} + (1 - p)n = O(n).$$

Busca Sequencial - Arquivo Ordenado

Arquivo ordenado (ordem crescente) pela chave primária.

Algorithm 2: Busca sequencial - arquivo ordenado.

entrada: Arquivo A com n registros em ordem crescente de chaves;
valor k a ser procurado

saida : Índice do registro no arquivo cuja chave é v (caso exista) ou
 -1 (caso contrário)

```
1  $i \leftarrow 1$ ;  
2 while  $(i \leq n) \wedge (A[i].k < k)$  do  
3    $i \leftarrow i + 1$ ;  
4 if  $(i \leq n) \wedge (A[i] = k)$  then  
5   return  $i$   
6 return  $-1$ ;
```

Complexidade

Algoritmo	Busca com Sucesso		Busca sem Sucesso	
	Melhor Caso	Pior Caso	Melhor Caso	Pior Caso
Busca Sequencial (não ordenado)	$O(1)$	$O(n)$	$O(n)$	
Busca Sequencial (ordenado)	$O(1)$	$O(n)$	$O(1)$	$O(n)$

Complexidade - Busca Sequencial (caso médio, arq. ordenado)

Modelo probabilístico

- ▶ A probabilidade de se buscar um registro existente no arquivo A é p
- ▶ Todos os registros de A têm mesma probabilidade de serem buscados

$$\forall i, j \in \{1, \dots, n\}, \forall k \in A (\Pr\{k = A[i].k\} = \Pr\{k = A[j].k\} = p/n)$$

- ▶ A probabilidade de se buscar um registro inexistente com chave maior que $A[i].k$ é a mesma para se busca um registro inexistente com chave maior que $A[j].k$

$$\forall i, j \in \{1, \dots, n\}, \forall r.k \notin A \left(\Pr\{k > A[i].k\} = \Pr\{k > A[j].k\} = \frac{1-p}{n} \right)$$

O custo médio C_n para se buscar um registro qualquer num arquivo com n registros é

$$C_n = \sum_{i=1}^n \frac{p}{n} \times i + \sum_{i=1}^n \frac{1-p}{n} \times n = \frac{p(n+1)}{2} + (1-p)n = O(n).$$

Busca por particionamentos sucessivos

Algorithm 3: Busca por particionamento sucessivos

entrada: Arquivo ordenado A por chave primária com n registros; valor k a ser buscado

saida : Índice do registro no arquivo cuja chave é k (caso exista) ou -1 (caso contrário)

```
1  $a \leftarrow 1$ ;  $b \leftarrow n$ ;  
2 while  $a \leq b$  do  
3    $i \leftarrow \text{nextPos}(k, a, b)$  ; /* deve garantir que  $a \leq i \leq b$  */  
4   if  $v = A[i].k$  then  
5     return  $i$ ;  
6   else if  $k > A[i].k$  then  
7      $a \leftarrow i + 1$ ;  
8   else  
9      $b \leftarrow i - 1$ ;  
10 return  $-1$ 
```

► Qual seria uma possibilidade para a função **nextPos**?

$$\text{nextPos}(k, a, b) \leftarrow \left\lfloor \frac{a + b}{2} \right\rfloor$$

Complexidade

Algoritmo	Busca com Sucesso		Busca sem Sucesso	
	Melhor Caso	Pior Caso	Melhor Caso	Pior Caso
Busca Sequencial (não ordenado)	$O(1)$	$O(n)$	$O(n)$	
Busca Sequencial (ordenado)	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Busca Binária	$O(1)$	$O(\log n)$	$O(\log n)$ - obs.1	

Obs. 1: Pode-se facilmente alterar o algoritmo para o melhor caso da busca binária sem sucesso ser $O(1)$

Caso médio: Qual seria a complexidade de busca binária no caso médio?

O que aconteceria se alterássemos a função **nextPos(k, a, b)** para que ela retornasse uma posição qualquer no intervalo $[a, b]$? Ou seja:

$$\text{nextPos}(k, a, b) = \text{random}(a, b)$$

O algoritmo continuaria correto?

A complexidade (pior caso, melhor caso, caso médio) continuaria a mesma?

Busca por Interpolação

Ideia básica: interpolar os valores das chaves, similar à busca de palavras num dicionário

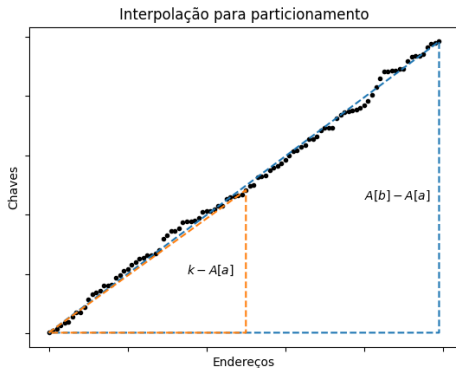


Figura: Interpolação para um conjunto de registros em ordem crescente de chaves, com valores distribuídos uniformemente ao longo do arquivo.

Busca por Interpolação

Tentativa de se escolher uma posição mais apropriada:

- Função de interpolação (funcionaria com o algoritmo apresentado?):

$$\text{nextPos}(k, a, b) \leftarrow \left\lceil a + \frac{(k - A[a].k)(b - a)}{A[b].k - A[a].k} \right\rceil \quad (1)$$

- Função alternativa:

$$\text{nextPos}(k, a, b) = \begin{cases} a & \text{se } a = b \\ \max \left\{ a, \min \left\{ b, \left\lceil a + \frac{(k - A[a].k)(b - a)}{A[b].k - A[a].k} \right\rceil \right\} \right\} & \text{se } a \neq b \end{cases} \quad (2)$$

Busca por Interpolação

Usando a função (1), o algoritmo deve ser levemente alterado para acomodar o caso de busca sem sucesso (valor fora da faixa de valores entre as posições i e f).

Algorithm 4: Busca por Interpolação

entrada: Arquivo **ordenado** por chave primária com n registros; valor k a ser buscado

saida : Índice do registro no arquivo cuja chave é k (caso exista) ou -1 (caso contrário)

```
1  $a \leftarrow 1; b \leftarrow n;$ 
2 while  $A[a].k < k \leq A[b].k$  do
3    $i \leftarrow \left\lceil a + \frac{(k - A[a].k)(b - a)}{A[b].k - A[a].k} \right\rceil;$ 
4   if  $k = A[i].k$  then  $a \leftarrow i;$ 
5   else if  $k > A[i].k$  then  $a \leftarrow i + 1;$ 
6   else  $b \leftarrow i - 1;$ 
7 if  $A[a].k = k$  then return  $a;$ 
8 return  $-1;$ 
```

Busca por Interpolação - Exemplo

Procura-se por $k = 26$:

1	2	3	4	5	6	7
6	12	15	25	26	35	40
a					b	

Passos:

$$1) i \leftarrow \left\lceil 1 + \frac{(26-6)(7-1)}{(40-6)} \right\rceil = 5$$

O valor da chave na posição 5 é igual a 26, ou seja, $A[5].k = v$

Achou com apenas uma comparação!

Quantas comparações seriam necessárias na busca binária?

Busca por Interpolação - Pior Caso

Procura-se por $k = 97$:

1	2	3	4	5
1	97	98	99	100
a				b

Passos:

$$1) i \leftarrow \left\lceil 1 + \frac{(97-1)(5-1)}{(100-1)} \right\rceil = 5$$

1	2	3	4	5
1	97	98	99	100
a				i=b

$$A[5].k > k: b \leftarrow i - 1$$

Busca por Interpolação - Pior Caso

Passos:

2)

1	2	3	4	5
1	97	98	99	100
a			i=b	

$$i \leftarrow \left\lceil 1 + \frac{(97-1)(4-1)}{(99-1)} \right\rceil = 4$$

$A[4].k > k: b \leftarrow i - 1$

3)

1	2	3	4	5
1	97	98	99	100
a		i=b		

$$i \leftarrow \left\lceil 1 + \frac{(97-1)(3-1)}{(98-1)} \right\rceil = 3$$

$A[3].k > k: b \leftarrow i - 1$

Busca por Interpolação - Pior Caso

Passos:

4)

1	2	3	4	5
1	97	98	99	100
a	i=b			

$$i \leftarrow \left\lceil 1 + \frac{(97-1)(2-1)}{(97-1)} \right\rceil = 2$$

$A[2].k = k$: Achou! Com $n - 1$ passos! **$O(n)$**

Complexidade

Algoritmo	Busca com Sucesso		Busca sem Sucesso	
	Melhor Caso	Pior Caso	Melhor Caso	Pior Caso
Busca Sequencial (não ordenado)	$O(1)$	$O(n)$	$O(n)$	
Busca Sequencial (ordenado)	$O(1)$	$O(n)$	$O(1)$	$O(n)$
Busca Binária	$O(1)$	$O(\log n)$	$O(\log n)$ - obs.1	
Interpolação	$O(1)$	$O(n)$	$O(1)$	$O(n)$

Obs. 1: Pode-se facilmente alterar o algoritmo para o melhor caso da busca binária sem sucesso ser $O(1)$

Obs. 2: A busca por interpolação possui caso médio $O(\log \log n)$

Ilustrações Comparativas

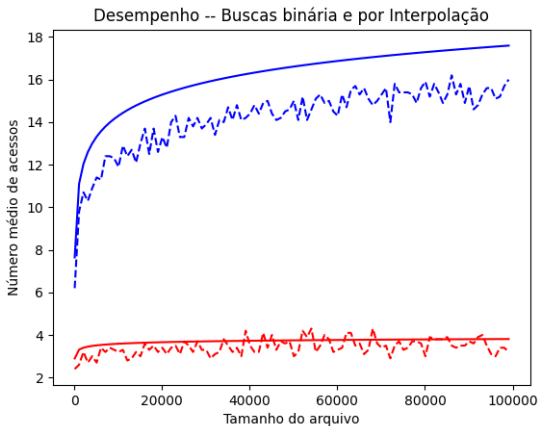


Figura: Número médio de acessos para se buscar um registro existente num arquivo usando pesquisa binária (azul) e por interpolação (vermelho). Linhas sólidas indicam os valores das funções $\log_2 n$ e $\log_2 \log_2 n$. Registros com chaves distribuídas uniformemente ao longo do arquivo.

Ilustrações Comparativas

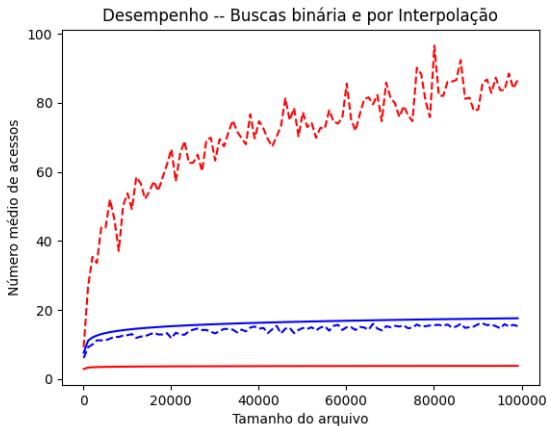


Figura: Número médio de acessos para se buscar um registro existente num arquivo usando pesquisa binária (azul) e por interpolação (vermelho). Linhas sólidas indicam os valores das funções $\log_2 n$ e $\log_2 \log_2 n$. As chaves dos registros estão dispostas de acordo com uma distribuição Exponencial, com parâmetro $\lambda = 1$.

Arquivos Sequenciais Auto-Organizados

- ▶ Manter arquivo em ordem física de chaves é custoso. Quais alternativas?
- ▶ E se registros com maior probabilidade de serem pesquisados estiverem no início:
 - ▶ **Mover para frente**: ao acessar $A[i]$, colocar $A[i]$ na primeira posição, deslocando os demais registros à direita
 - ▶ **Transpor**: ao acessar $A[i]$ ($i > 1$), trocar sua posição com $A[i - 1]$
 - ▶ **Contador de acesso**: mantém a ordem de acesso proporcional ao número de acessos a cada registro.
- ▶ Qual o custo-benefício? Outras possibilidades para auto-organização?

1. Qual seria o comportamento de busca binária se a condição do laço fosse alterada de $a \leq b$ para $a < b$? Analise as consequências desta modificação para busca binária e para busca por interpolação. Se necessário, adapte o algoritmo e as funções `nextPos`.
2. Como busca por interpolação oferece vantagens sobre busca binária apenas quando os valores das chaves estão uniformemente distribuídos ao longo do arquivo, elabore um algoritmo que mescla ambas as estratégias, adaptando-se durante a busca.
3. Usando um exemplo com 10 registros, forneça o pior caso para:
 - 3.1 busca binária
 - 3.2 busca por interpolação

4. Encontre a expressão que fornece o número médio de acessos para recuperar um registro existente num arquivo com n registros usando busca binária. Considere que qualquer registro do arquivo tem igual probabilidade a ser buscado.
 - ▶ Dica: observe o número de acessos para cada registro; 1 registro é buscado com 1 acesso; 2 registros são buscados com 2 acessos; 4 registros são buscados com 3 acessos; e assim sucessivamente.
5. Derive a expressão que fornece o número médio de acessos do Algoritmo 3 caso a linha 3 fosse substituída por $\text{nextPos}(v, a, b) = \text{random}(a, b)$. Considere que $\text{random}(a, b)$ retorna um valor inteiro no intervalo $[a, b]$.

6. Compare experimentalmente o comportamento da busca por particionamento sucessivo considerando que
- 0.1 $\text{nextPos}(v, a, b) = \text{random}(a, b)$.
 - 0.2 $\text{nextPos}(v, a, b) = a + \lceil (a + b)/3 \rceil$.
 - 0.3 $\text{nextPos}(v, a, b) = a + \lceil (a + b)/10 \rceil$.
7. Ilustre o comportamento de busca binária e busca por interpolação para tamanhos de arquivos $n \in [1000, 100000]$, considerando que os valores das chaves dos registros estão distribuídos de acordo com $\mathcal{N}(1000, 100)$, uma distribuição Normal com média $\mu = 1000$ e variância $\sigma^2 = 100$. Recomenda-se que haja ao menos 10 repetições do experimento para cada valor considerado de n . O gráfico a ser exibido deve adotar a média dos valores obtidos para cada valor de n .